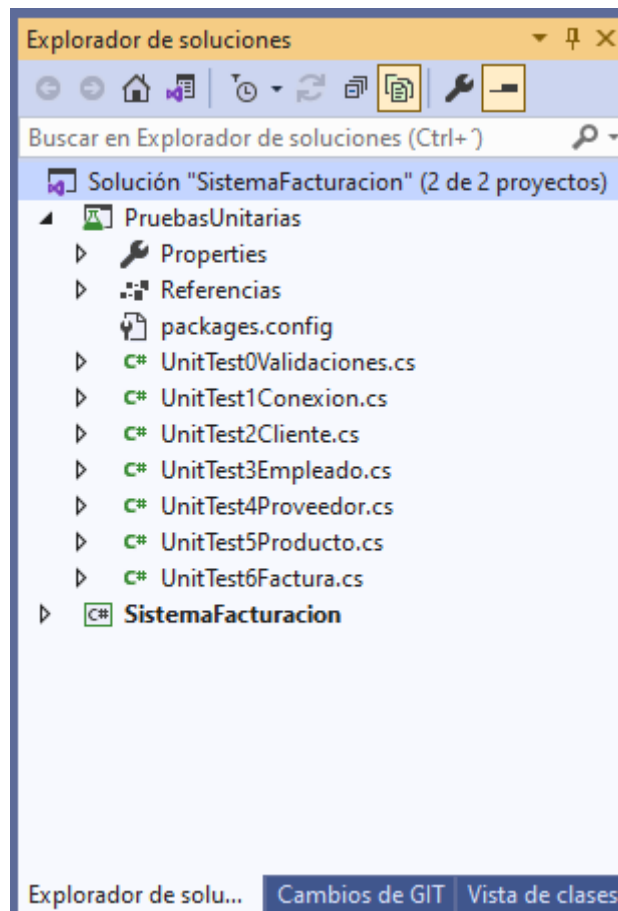


PRUEBAS UNITARIAS

Se realizaron una prueba de unitarias para comprobar el correcto funcionamiento de las diferentes funciones que pueden ser realizadas dentro del sistema. Para ejecutar cada una de las actividades de tal forma que se simule el trabajo real se definieron pruebas para cada componente. En la siguiente imagen se puede visualizar la estructura de la solución de “MsTestFramework” denominada “PruebasUnitarias”.



Existe una clase de pruebas unitarias específicas para cada una de las entidades de alta importancia en el sistema. Entre ellas, cliente, empleado, **proveedor**, producto y factura. Además, de pruebas de conexión y validación de datos como la cédula y contraseñas.

Se podrá visualizar que cada una de las pruebas unitarias tiene un número específico, esto es para garantizar que se ejecute en un orden específico y poder determinar hasta donde se ejecutarán correctamente las pruebas. Esta misma lógica se aplicará a los métodos de pruebas desarrollados para cada clase *UnitTest*.

Con lo ya especificado se procede a especificar el contenido de cada una de estas clases y el objetivo de cada una de las pruebas.

1. PRUEBAS UNITARIAS PARA VALIDACIÓN DE DATOS

La implementación de las pruebas unitarias para validación de datos en el proyecto se centra en evaluar el cumplimiento de las restricciones de la cédula y contraseñas seguras, como se muestra a continuación.

```
[TestClass]
0 referencias
public class UnitTest0Validaciones
{
    [DataTestMethod]
    [DataRow("Password123!")]
    [DataRow("Contra:2023")]
    [DataRow("ContraseñaD2#")]
    0 referencias
    public void MT_Contrasenia_valida(string contrasenia)
    {
        Assert.IsTrue(Validaciones.esContraseniaValida(contrasenia));
    }

    [DataTestMethod]
    [DataRow("Password123")]
    [DataRow("Contra2023")]
    [DataRow("ContraseñaD2")]
    [DataRow("!223123124@")]
    0 referencias
    public void MT_Contrasenia_no_valida(string contrasenia)
    {
        Assert.IsFalse(Validaciones.esContraseniaValida(contrasenia));
    }
}
```

Los dos métodos de prueba presentados en la ilustración anterior evalúan una serie de contraseñas para determinar su validez según los métodos definidos en la clase Validaciones, misma que contiene métodos estáticos. Con esto validamos si los datos que se están enviando al sistema son válidos o no.

```
[DataTestMethod]
[DataRow("1234569874")]
[DataRow("2350703385")]
[DataRow("1705236521")]
[DataRow("1705236522")]
[DataRow("1705236523")]
[DataRow("1725231524")]
0 referencias
public void MT_Cedula_valida(string cedula)
{
    Assert.IsTrue(Validaciones.esCedulaValida(cedula));
}

[DataTestMethod]
[DataRow("123456987")]
[DataRow("23507033865")]
[DataRow("17052E6521")]
[DataRow("1005236522")]
[DataRow("170Q23652E")]
[DataRow("17231524")]
0 referencias
public void MT_Cedula_no_valida(string cedula)
{
    Assert.IsFalse(Validaciones.esCedulaValida(cedula));
}
```

Por su contraparte los dos métodos presentados en esta figura verifican un conjunto de cédulas de identidad para determinar su validez. Eso se realiza mediante las expresiones regulares teniendo una buena verificación antes de que los datos se envíen al sistema.

2. PRUEBAS UNITARIAS DE CONEXIÓN

La prueba unitaria para la conexión tiene como objetivo comprobar que la conexión pueda abrirse y cerrarse correctamente evitando posibles excepciones debido a una mala gestión de la apertura y cierre de las conexiones.

```
[TestClass]
0 referencias
public class UnitTest1Conexion
{
    [TestMethod]
    0 referencias
    public void TM_Conexion_abierta()
    {
        Conexion con = new Conexion();
        con.AbrirConexion();

        Assert.IsTrue(con.ConexionAbierta());
        con.CerrarConexion();
    }

    [TestMethod]
    0 referencias
    public void TM_Conexion_cerrada_sin_iniciar()
    {
        Conexion con = new Conexion();
        Assert.IsFalse(con.ConexionAbierta());
    }

    [TestMethod]
    0 referencias
    public void TM_Conexion_cerrada()
    {
        Conexion con = new Conexion();
        con.AbrirConexion();
        con.CerrarConexion();

        Assert.IsFalse(con.ConexionAbierta());
    }
}
```

3. PRUEBAS UNITARIAS PARA ENTIDAD CLIENTES

Las pruebas unitarias para cada una de las entidades contienen el flujo completo para insertar, modificar, buscar, obtener y eliminar un registro de la base de datos. A continuación, se muestra los métodos de prueba desarrollados para la entidad cliente.

```
[TestMethod]
0 | Referencias
public void TM_1_Insertar_cliente()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = new Cliente();

        cliente.Cedula_cliente = "0000000000";
        cliente.Apellidos = "Apellidos";
        cliente.Nombres = "Nombres";

        Respuesta r = clienteCtrl.insertarCliente(cliente);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

Este método de prueba simplemente crea un cliente con una serie de datos ya definidos esperando que el resultado o respuesta sea un valor de verdad, si es así, entonces se cumplirá correctamente el método.

```
[TestMethod]
0 | Referencias
public void TM_2_Buscar_cliente()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "cedula_cliente"));
        filtros.Add(new Filtros("texto_buscar", "0000000000"));

        List<Cliente> lstCliente = clienteCtrl.buscarClientes(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstCliente.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}
```

Por otro lado, el método de prueba 2 de la entidad Cliente, realiza una búsqueda del cliente recién creado, como resultado se espera que la lista de clientes resultantes de la búsqueda sea equivalente a 1, especialmente debido a que es una búsqueda por cédula.

```

[TestMethod]
0 | 0 referencias
public void TM_3_Get_cliente_por_cedula()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientPorCedula("0000000000");

        Assert.IsNotNull(cliente);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El método de prueba 3 obtiene el mismo cliente registrado, pero ahora mediante un método específico que garantiza retornar un único objeto de tipo cliente si es que existe. Por esa razón su verificación es considerando si el objeto no es nulo.

```

[TestMethod]
0 | 0 referencias
public void TM_4_Get_cliente_por_id()
{
    bool encontrado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientPorCedula("0000000000");

        if (cliente != null)
        {
            Cliente c = clienteCtrl.getClientPorId(cliente.Id_cliente);
            encontrado = c != null;
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Por contraparte al método anterior, este busca al cliente por el id asociado, debido a que se desconoce el que sea asignado por la base de datos (a pesar de ser incremental), primero se realiza una búsqueda por la cédula que devolverá los datos del cliente y el id se enviará para verificar si existe realmente.

```

[TestMethod]
0 | Referencias
public void TM_5_Modificar_cliente()
{
    bool modificado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000000");

        if (cliente != null)
        {
            cliente.Nombres = "NombresMod";
            cliente.Apellidos = "ApellidosMod";

            Respuesta r = clienteCtrl.modificarCliente(cliente);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El método 5 de la entidad Clientes, realiza la modificación del cliente creado en el método 1. Simplemente cambia el nombre y apellido para luego evaluar si realmente existió alguna modificación. Además, para ello se obtiene el objeto completo mediante la obtención del cliente por su cédula.

```

[TestMethod]
0 | 0 referencias
public void TM_6_Eliminar_cliente()
{
    bool eliminado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000000");

        if (cliente != null)
        {
            Respuesta r = clienteCtrl.eliminarCliente(cliente.Id_cliente);
            eliminado = r.Completado;
        }

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Finalmente, el último método de prueba de la entidad Cliente, es eliminar, en ella sigue el proceso de los métodos anteriores; que es, obtener el cliente por la cédula y ejecutar la acción correspondiente a eliminarlo.

4. PRUEBAS UNITARIAS PARA ENTIDAD EMPLEADO

Al igual que los métodos de prueba definidos en las pruebas unitarias de Cliente, en esta entidad se desarrollan las transacciones de consulta y de manipulación de datos. Es importante mencionar que para crear un empleado se requiere definir las credenciales y roles correspondientes según lo especificado en la clase Usuario.

```
public void TM_1_Insertar_empleado()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();

        //Verificar si ya está registrado
        Cliente cliente = clienteCtrl.getClientPorCedula("0000000001");
        if (cliente != null)
        {
            clienteCtrl.eliminarCliente(cliente.IdPersona);
        }

        Empleado empleado = new Empleado();
        empleado.Cedula = "0000000001";
        empleado.Apellidos = "Apellidos";
        empleado.Nombres = "Nombres";
        empleado.Fecha_Nacimiento = new DateTime(1955, 1, 1).ToString();

        Usuario usuario = new Usuario();
        usuario.Tipo_usuario.Id_tipo_usuario = 1;
        usuario.Nombre_usuario = "_Usuario_";
        usuario.Contrasenia = "123";
        usuario.Activo = true;

        Respuesta r = empleadoCtrl.insertarEmpleado(empleado, usuario);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception e)
    {
        Assert.IsTrue(false);
        throw new Exception(e.Message);
    }
}
```

El primer método de prueba realiza la inserción de un empleado con parámetros ya definidos y con la inicialización de un objeto de tipo usuario para establecer las credenciales asociadas al mismo.

```

[TestMethod]
0 referencias
public void TM_2_Get_empleado_por_cedula()
{
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        Assert.IsNotNull(empleado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Por otro lado, el segundo método de prueba realiza la búsqueda del empleado que coincida con la cédula especificada y creada en el método anterior. creado en el método anterior, como resultado se espera que el objeto empleado no sea nulo.

```

[TestMethod]
0 referencias
public void TM_3_Get_empleado_por_id()
{
    bool encontrado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        if (empleado != null)
        {
            Empleado e = empleadoCtrl.getEmpleadoPorId(empleado.Id_empleado);
            encontrado = (e != null);
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El tercer método de prueba permite obtener los datos de un empleado según el id que tenga asociado. Para ello, se utiliza el método de obtener por cédula y así enviar el parámetro requerido al método. Como resultado se espera que el objeto no sea nulo y se haya encontrado.

```

[TestMethod]
0 referencias
public void TM_4_Modificar_empleado()
{
    bool modificado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        if (empleado != null)
        {
            empleado = empleadoCtrl.getEmpleadoUsuarioPorIdEmpleado(empleado.Id_empleado);

            empleado.Nombres = "NombresMod";
            empleado.Apellidos = "ApellidosMod";
            empleado.Usuario.Contrasenia = "123";

            Respuesta r = empleadoCtrl.modificarEmpleado(empleado, empleado.Usuario);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El método de prueba 4, modifica los datos de un empleado, cambiando su nombre, apellido y estableciendo su contraseña, puesto que es requerido para validar que los datos sean modificados.

```

[TestMethod]
0 referencias
public void TM_6_Modificar_contrasenia_empleado()
{
    try
    {
        UsuarioCtrl usuarioCtrl = new UsuarioCtrl();
        Respuesta r = usuarioCtrl.modificarContrasenia("_Usuario_", "123", "1234");

        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Así mismo, para modificar la contraseña se requiere de la contraseña actual y la nueva contraseña. Por esta razón se requiere evaluar la validez del método. En los anteriores se

estableció que la contraseña es 123 por lo tanto se espera que el resultado de este método de prueba sea un valor de verdad (true).

```
[TestMethod]
0 referencias
public void TM_6_Modificar_empleado_contraseña_incorrecta()
{
    bool modificado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        if (empleado != null)
        {
            empleado = empleadoCtrl.getEmpleadoUsuarioPorIdEmpleado(empleado.Id_empleado);

            empleado.Nombres = "NombresMod";
            empleado.Apellidos = "ApellidosMod";
            empleado.Usuario.Contrasenia = "123";

            Respuesta r = empleadoCtrl.modificarEmpleado(empleado, empleado.Usuario);
            modificado = r.Completado;
        }

        Assert.IsFalse(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

También se evalúa la modificación de datos cuando la contraseña es incorrecta. Es decir, en el método de prueba anterior (5) se cambió la contraseña a 1234. Por lo tanto, este método será válido solo si la modificación de los datos no se realiza; o, en otras palabras, es falso.

```
[TestMethod]
0 referencias
public void TM_7_Eliminar_empleado()
{
    bool eliminado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        if (empleado != null)
        {
            Respuesta r = empleadoCtrl.eliminarEmpleado(empleado.Id_empleado);
            eliminado = r.Completado;
        }

        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientesPorCedula("0000000001");
        if (cliente != null)
        {
            clienteCtrl.eliminarCliente(cliente.IdPersona);
        }

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

Finalmente, se realiza el método de prueba para eliminar el registro con el que se trabajó durante el flujo de esta prueba unitaria. Se podrá visualizar que primero se obtiene el empleado y se elimina. Inmediatamente, se requiere del controlador de Cliente puesto que el empleado también es registrado como un cliente, y al eliminar el objeto de empleado es necesario eliminar el objeto de tipo Cliente.

6. PRUEBAS UNITARIAS PARA ENTIDAD PROVEEDOR

La nueva entidad proveedor lleva una relación directa con productos; por lo tanto, se realizarán y detallarán en primera instancia los métodos de consulta y de manipulación de datos de esta entidad.

```
[TestMethod]
0 | 0 referencias
public void TM_1_Insertar_proveedor()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = new Proveedor();

        proveedor.Cedula = "0000000002";
        proveedor.Apellidos = "Apellidos";
        proveedor.Nombres = "Nombres";
        proveedor.Email = "prueba_proveedor@gmail.com";
        proveedor.Fecha_Nacimiento = new DateTime(2002, 10, 12).ToString();

        Respuesta r = proveedorCtrl.insertarProveedor(proveedor);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

El primer método de prueba es la inserción del proveedor, en este se definen datos fijos como la cédula, apellidos, nombres, email y la fecha de nacimiento. Posterior a ello, se procede a realizar la inserción mediante el método correspondiente. Como resultado, se espera que se haya insertado el proveedor.

```

[TestMethod]
0 referencias
public void TM_2_Buscar_proveedor()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "cedula"));
        filtros.Add(new Filtros("texto_buscar", "0000000002"));

        List<Proveedor> lstCliente = proveedorCtrl.buscarProveedores(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstCliente.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}

```

El segundo método de prueba realiza la búsqueda de un proveedor por su cédula. En este se espera obtener un único objeto en la lista, mismo que fue ingresado en el método de prueba anterior.

```

[TestMethod]
0 referencias
public void TM_3_Get_proveedor_por_cedula()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        Assert.IsNotNull(proveedor);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Por otro lado, el método de prueba 3, realiza la búsqueda del mismo proveedor, pero ahora lo realiza específicamente por la cédula con la que fue creada. Como resultado, se espera que el objeto no sea nulo.

```

[TestMethod]
0 | 0 referencias
public void TM_4_Get_proveedor_por_id()
{
    bool encontrado = false;
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        if (proveedor != null)
        {
            proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);
            encontrado = proveedor != null;
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El cuarto método de prueba tiene como objetivo obtener datos del mismo proveedor creado, pero por el id correspondiente. Para ello, se usa el método de buscar por cédula y así poder obtener la cédula para enviarla como parámetro. Como resultado, se espera que se encuentre el proveedor.

```

[TestMethod]
✓ | 0 referencias
public void TM_5_Modificar_proveedor()
{
    bool modificado = false;
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        if (proveedor != null)
        {
            proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);
            proveedor.Nombres = "NombresMod";
            proveedor.Apellidos = "ApellidosMod";

            Respuesta r = proveedorCtrl.modificarProveedor(proveedor);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El quinto método de prueba de la entidad proveedor, obtiene principalmente los datos del proveedor creado y realiza cambios en el nombre y apellido. Luego, se espera que se haya modificado correctamente obteniendo un valor de verdad (true).

```

[TestMethod]
0 | 0 referencias
public void TM_6_Eliminar_proveedor()
{
    bool eliminado = false;
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        if (proveedor != null)
        {
            proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);

            Respuesta r = proveedorCtrl.eliminarProveedor(proveedor.IdPersona);
            eliminado = r.Completado;
        }

        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000002");
        if (cliente != null)
        {
            clienteCtrl.eliminarCliente(cliente.IdPersona);
        }
        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Finalmente, se realiza la eliminación del proveedor. Al igual que la entidad Empleado debido a que esto se trata de pruebas unitarias y no de un flujo en su completitud, posterior a la obtención de los datos del proveedor y su eliminación, es necesario eliminar los datos de la tabla que contiene todos los objetos de tipo Cliente o más específicamente Persona.

7. PRUEBAS UNITARIAS PARA ENTIDAD PRODUCTOS

La clase de pruebas unitarias para la entidad productos tenemos varios métodos que nos ayudan a realizar las operaciones básicas de CRUD para de esta manera verificar el correcto funcionamiento de los procesos.

En el siguiente método podemos observar que tenemos varios campos en los cuales están insertar producto con un proveedor determinado. Claro que este proveedor se lo crea en el instante para verificar la correcta relación de las tablas.

```
[TestMethod]
0 referencias
public void TM_1_Insertar_producto()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = new Proveedor();

        proveedor.Cedula = "0000000002";
        proveedor.Apellidos = "Apellidos";
        proveedor.Nombres = "Nombres";
        proveedor.Fecha_Nacimiento = new DateTime(2002, 10, 12).ToString();

        proveedorCtrl.insertarProveedor(proveedor);
        proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        ProductoCtrl productoCtrl = new ProductoCtrl();
        Producto producto = new Producto();

        producto.Nombre_producto = "00Producto00";
        producto.Iva = 0;
        producto.Precio_unitario = 0;
        producto.Proveedor = proveedor;

        Respuesta r = productoCtrl.insertarProducto(producto);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

En este método tenemos el proceso de buscar que lo realiza mediante filtros. Elegimos el texto que deseamos buscar seleccionando mediante el cual campo lo queremos hacer la búsqueda. Si los registros retornados son igual a uno es porque el método devolvió un resultado correcto caso contrario no.

```

[TestMethod]
0 referencias
public void TM_2_Buscar_producto()
{
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "nombre_producto"));
        filtros.Add(new Filtros("texto_buscar", "00Producto00"));

        List<Producto> lstProducto = productoCtrl.buscarProductoPaginacion(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstProducto.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
    }
}

```

En el siguiente método realizamos una operación similar a la anterior, pero con la diferencia que en esta estamos comparando si el producto que se ha buscado no es nulo es decir que existe dentro del sistema.

```

[TestMethod]
0 referencias
public void TM_3_Get_producto()
{
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "nombre_producto"));
        filtros.Add(new Filtros("texto_buscar", "00Producto00"));

        Producto producto = productoCtrl.buscarProductoPaginacion(1, 30, Filtros.filtros_a_xml("Filtros", filtros))[0];
        producto = productoCtrl.getProducto(producto.Id_producto);

        Assert.IsNotNull(producto);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

En el siguiente método estamos realizando un conjunto de operaciones, búsqueda y modificación de datos. En este método primero se obtiene un producto de la base de datos mediante el método de búsqueda. De acuerdo con esto se selecciona la primera posición del producto devuelto y con ello se modifican los campos nombres, IVA y precio unitario. Finalmente, el producto se actualiza mediante el método modificarProducto.

```

[TestMethod]
0 | 0 referencias
public void TM_4_Modificar_producto()
{
    bool modificado = false;
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        List<Producto> lstProducto = productoCtrl.buscarProducto("00Producto00");

        if (lstProducto.Count > 0)
        {
            Producto producto = lstProducto[0];
            producto.Nombre_producto = "00Producto00Mod";
            producto.Iva = 0;
            producto.Precio_unitario = 0;

            Respuesta r = productoCtrl.modificarProducto(producto);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

En el siguiente proceso vemos el método de eliminación de producto en la cual primero obtenemos el producto que queremos eliminar y posteriormente verificamos si existe si la lista tiene un tamaño mayor a cero. Luego lo eliminamos si existe. Cabe resaltar que en esta verificación comprobamos si la eliminación de realizo correctamente es decir si retorna un resultado verdadero.

```

[TestMethod]
0 referencias
public void TM_5_Eliminar_producto()
{
    bool eliminado = false;
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        List<Producto> lstProducto = productoCtrl.buscarProducto("00Producto00Mod");

        if (lstProducto.Count > 0)
        {
            Producto producto = lstProducto[0];

            Respuesta r = productoCtrl.eliminarProducto(producto.Id_producto);
            eliminado = r.Completado;
        }

        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);
        proveedorCtrl.eliminarProveedor(proveedor.IdPersona);

        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000002");
        clienteCtrl.eliminarCliente(cliente.IdPersona);

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

8. PRUEBAS UNITARIAS PARA ENTIDAD FACTURA

Para las pruebas unitarias de Factura, se utilizó la notación initialize para registrar un cliente, **proveedor**, producto y empleado para poder realizar las acciones de consulta y manipulación de datos de la factura. A continuación, se muestra el proceso.

```

[ClassInitialize]
0 referencias
public static void Initialize(TestContext context)
{
    try
    {
        //Insertar un empleado y usuario para las pruebas
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();

        Empleado empleado = new Empleado();
        empleado.Cedula = "0000000001";
        empleado.Apellidos = "Apellidos";
        empleado.Nombres = "Nombres";
        empleado.Fecha_Nacimiento = new DateTime(1955, 1, 1).ToString();

        Usuario usuario = new Usuario();
        usuario.Tipo_usuario.Id_tipo_usuario = 1;
        usuario.Nombre_usuario = "_Usuario_";
        usuario.Contrasenia = "123";
        usuario.Activo = true;

        empleadoCtrl.insertarEmpleado(empleado, usuario);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar empleado: {ex.Message}");
    }

    //Insertar un cliente
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = new Cliente();

        cliente.Cedula = "0000000000";
        cliente.Apellidos = "Apellidos";
        cliente.Nombres = "Nombres";

        clienteCtrl.insertarCliente(cliente);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar cliente: {ex.Message}");
    }

    //Insertar un producto
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = new Proveedor();

        proveedor.Cedula = "0000000002";
        proveedor.Apellidos = "Apellidos";
        proveedor.Nombres = "Nombres";
        proveedor.Fecha_Nacimiento = new DateTime(2002, 10, 12).ToString();

        proveedorCtrl.insertarProveedor(proveedor);
        proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        id_proveedor = proveedor.IdProveedor;

        ProductoCtrl productoCtrl = new ProductoCtrl();
        Producto producto = new Producto();

        producto.Nombre_producto = "00Producto00";
        producto.Iva = 0;
        producto.Precio_unitario = 0;
        producto.Proveedor = proveedor;

        productoCtrl.insertarProducto(producto);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}

```

Llegados a este punto se comprobó el funcionamiento de todas las entidades necesarias para llegar al proceso de facturación. Por esta razón, se inicializa la clase con un conjunto de objetos requeridos para su desarrollo.

```
[TestMethod]
0 referencias
public void TM_1_Insertar_factura()
{
    try
    {
        FacturaCtrl facturaCtrl = new FacturaCtrl();

        Factura factura = new Factura();
        FacturaDetalle fDetalle = new FacturaDetalle();

        EmpleadoCtrl eCtrl = new EmpleadoCtrl();
        Empleado empleadoPrueba = eCtrl.getEmpleadoPorCedula("000000001");

        ProductoCtrl pCtrl = new ProductoCtrl();
        Producto productoPrueba = pCtrl.buscarProducto("00Producto00")[0];

        ClienteCtrl cCtrl = new ClienteCtrl();
        Cliente clientePrueba = cCtrl.getClientPorCedula("000000000");

        factura.Cliente = clientePrueba;
        factura.Empleado = empleadoPrueba;
        fDetalle.Producto = productoPrueba;
        fDetalle.Cantidad = 0;

        factura.Factura_Detalle.Add(fDetalle);
        Respuesta r = facturaCtrl.insertarFactura(factura);

        id_factura = int.Parse(r.Id);
        id_empleado = empleadoPrueba.Id_empleado;
        id_producto = productoPrueba.Id_producto;

        Assert.IsTrue(r.Completado);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
        throw new Exception($"Error al insertar producto: {ex.ToString()}");
    }
}
```

La inserción de la factura es el primer método de prueba definido para la entidad Factura, en ella se realiza la búsqueda de cada uno de los objetos según lo establecido en la inicialización (mostrada en la ilustración anterior). Una vez se obtienen los datos requeridos, se asignan los valores al objeto de factura detalle y se agregan a la Factura. Así, como resultado se espera que se haya registrado la Factura obtienen un valor de verdad (true).

```

[TestMethod]
0 referencias
public void TM_2_Buscar_facturas()
{
    try
    {
        FacturaCtrl facturaCtrl = new FacturaCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "id_factura"));
        filtros.Add(new Filtros("texto_buscar", id_factura.ToString()));
        filtros.Add(new Filtros("fecha_inicio", DateTime.Now.ToString("MM-dd-yyyy")));
        filtros.Add(new Filtros("fecha_fin", DateTime.Now.ToString("MM-dd-yyyy")));

        List<Factura> facturas = facturaCtrl.buscarFacturas(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(facturas.Count == 1);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El segundo y último método de prueba de la entidad Factura es realizar la búsqueda de la factura recién ingresada. Para ello, busca el id de la factura registrada, y que la fecha de registro sea en el momento en el que se ejecutan las pruebas. Luego se ejecuta el método solicitando la primera página de la paginación con 30 elementos, y se espera que la longitud de la lista de objetos de tipo factura sea igual a 1.

[ClassCleanup]

0 referencias

```
public static void Cleanup()
{
    //Eliminar el cliente, producto, empleado y factura utilizado en las pruebas
    //Eliminar factura y su detalle
    FacturaCtrl facturaCtrl = new FacturaCtrl();
    facturaCtrl.eliminarFactura(id_factura);

    //Eliminar el empleado
    EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
    empleadoCtrl.eliminarEmpleado(id_empleado);

    //Eliminar el producto
    ProductoCtrl productoCtrl = new ProductoCtrl();
    productoCtrl.eliminarProducto(id_producto);

    //Eliminar el proveedor
    ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
    proveedorCtrl.eliminarProveedor(id_proveedor);

    //Eliminar el cliente
    ClienteCtrl clienteCtrl = new ClienteCtrl();
    Cliente clientePrueba = clienteCtrl.getClientePorCedula("0000000000");
    clienteCtrl.eliminarCliente(clientePrueba.IdPersona);

    clientePrueba = clienteCtrl.getClientePorCedula("0000000001");
    clienteCtrl.eliminarCliente(clientePrueba.IdPersona);

    clientePrueba = clienteCtrl.getClientePorCedula("0000000002");
    clienteCtrl.eliminarCliente(clientePrueba.IdPersona);
}
```

Finalmente, para eliminar los datos se ejecuta la función Cleanup(), misma que se ejecuta automáticamente al terminar de evaluar cada una de las pruebas unitarias de la clase de Factura.

Así, al ejecutar las pruebas unitarias, se obtienen los siguientes resultados.

Explorador de pruebas		UnitTest5Producto.cs	UnitTest4Proveedor.cs
		33	33
		0	0
Prueba	Duración	Rasgos	M...
✓ PruebasUnitarias (33)	3,9 s		
✓ PruebasUnitarias (33)	3,9 s		
✓ UnitTest0Validaciones (4)	85 ms		
✓ UnitTest1Conexion (3)	420 ms		
✓ UnitTest2Cliente (6)	824 ms		
✓ UnitTest3Empleado (7)	743 ms		
✓ UnitTest4Proveedor (6)	638 ms		
✓ UnitTest5Producto (5)	966 ms		
✓ UnitTest6Factura (2)	197 ms		

O de forma específica.

Explorador de pruebas		UnitTest5Producto.cs	UnitTest4Proveedor.cs
		33	33
		0	0
Prueba	Duración	Rasgos	M...
✓ PruebasUnitarias (33)	3,9 s		
✓ PruebasUnitarias (33)	3,9 s		
✓ UnitTest0Validaciones (4)	85 ms		
✓ MT_Cedula_no_valida	72 ms		
✓ MT_Cedula_valida	< 1 ms		
✓ MT_Contrasenia_no_valida	13 ms		
✓ MT_Contrasenia_valida	< 1 ms		
✓ UnitTest1Conexion (3)	420 ms		
✓ TM_Conexion_abierta	418 ms		
✓ TM_Conexion_cerrada	2 ms		
✓ TM_Conexion_cerrada_sin_i...	< 1 ms		
✓ UnitTest2Cliente (6)	824 ms		
✓ TM_1_Insertar_cliente	381 ms		
✓ TM_2_Buscar_cliente	108 ms		
✓ TM_3_Get_cliente_por_cedula	22 ms		
✓ TM_4_Get_cliente_por_id	40 ms		
✓ TM_5_Modificar_cliente	194 ms		
✓ TM_6_Eliminar_cliente	79 ms		
✓ UnitTest3Empleado (7)	743 ms		
✓ TM_1_Insertar_empleado	95 ms		
✓ TM_2_Get_empleado_por_c...	43 ms		
✓ TM_3_Get_empleado_por_id	43 ms		
✓ TM_4_Modificar_empleado	133 ms		
✓ TM_5_Modificar_contraseni...	73 ms		
✓ TM_6_Modificar_empleado_...	62 ms		
✓ TM_7_Eliminar_empleado	294 ms		
✓ UnitTest4Proveedor (6)	638 ms		
✓ TM_1_Insertar_proveedor	181 ms		
✓ TM_2_Buscar_proveedor	23 ms		
✓ TM_3_Get_proveedor_por_c...	26 ms		
✓ TM_4_Get_proveedor_por_id	40 ms		
✓ TM_5_Modificar_proveedor	151 ms		
✓ TM_6_Eliminar_proveedor	217 ms		
✓ UnitTest5Producto (5)	966 ms		
✓ TM_1_Insertar_producto	269 ms		
✓ TM_2_Buscar_producto	44 ms		
✓ TM_3_Get_producto	92 ms		
✓ TM_4_Modificar_producto	150 ms		
✓ TM_5_Eliminar_producto	411 ms		
✓ UnitTest6Factura (2)	197 ms		
✓ TM_1_Insertar_factura	165 ms		
✓ TM_2_Buscar_facturas	32 ms		